

Dealership data platform

A Bronze / Silver / Gold lakehouse with LLM-style intake agents, built for a fictional Nigerian used-car import dealership

Prepared for: technical & investor stakeholders Author: Dexter Date: July 2026

What this is

This project is an end-to-end simulation of a real operating problem: a used-car dealership that imports vehicles from overseas auctions, repairs and cleans them locally, and sells them through a small sales team, currently tracking all of this across WhatsApp messages, paper receipts, and spreadsheets. The project builds the data platform that business would need to run itself on real numbers instead of memory: a governed lakehouse that ingests messy inputs, structures them for analysis, and reports back the metrics that matter — profit per vehicle, time to sale, and where inventory is getting stuck.

The dataset is synthetic (no real dealership's data is used), but the pipeline, schema design, and automation logic are built the way they would be for a live business, so the same architecture generalizes directly to a real deployment.

VEHICLES TRACKED	TOTAL REVENUE	TOTAL PROFIT	AVG MARGIN
40	₦145.5M	₦34.6M	22.91%

Why it matters

For the business

Every vehicle's full lifecycle — purchase, shipping, customs, repair, cleaning, listing, sale — is tied together, so profit, cycle time, and bottlenecks are visible per vehicle and in aggregate, not buried across group chats and paper.

For the technical build

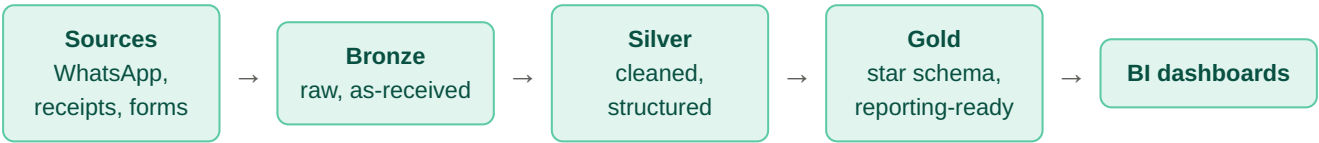
Demonstrates a complete lakehouse pattern: layered storage, dimensional modeling, idempotent automation, and a working ingestion path for unstructured inputs (chat messages, receipt photos) into governed tables.

What was built

Component	Description
Bronze layer	Raw landing zone for WhatsApp messages and receipt documents, unmodified as received
Silver layer	26 cleaned, structured tables spanning the full vehicle lifecycle from purchase to payment
Gold layer	Star schema (10 dimensions, 6 fact tables) built for reporting and dashboarding
Ingestion agents	Rule-based WhatsApp event extraction and simulated receipt OCR, both gated by a confidence-scored review queue
Executive dashboard	Mockup of the top-level KPI view: funnel, revenue, profit, margin, cycle time

Architecture

Data flows through three governed layers, a pattern known as the medallion architecture. Each layer has a specific job, so a bug or format change in one place doesn't propagate silently downstream.



Ingestion agents

Two intake paths simulate what an LLM-backed agent would do in production, without requiring a live model call for the portfolio build:

- **WhatsApp group agent** — classifies free-text messages into structured events (vehicle arrived, repair done, sold) with a confidence score. Low-confidence matches route to a human review queue rather than being silently accepted or dropped.
- **Receipt intake agent** — simulates OCR extraction of line items from receipt photos, with an injected error rate to model real-world OCR imperfection, again gated through the same review queue.

Technology stack

Layer	Technology	Why
Storage engine	Delta Lake 4.1.0	ACID transactions, schema enforcement, time travel — same engine used on Databricks
Compute	PySpark 4.1.0	Distributed SQL/DataFrame processing; code written to be Databricks-portable
Environment	Docker + Docker Compose	Reproducible local dev environment, avoids OS-specific Spark setup issues
Synthetic data	Faker	Realistic fake records standing in for real dealership data
Version control	Git / GitHub	Full commit history of the build, available for review

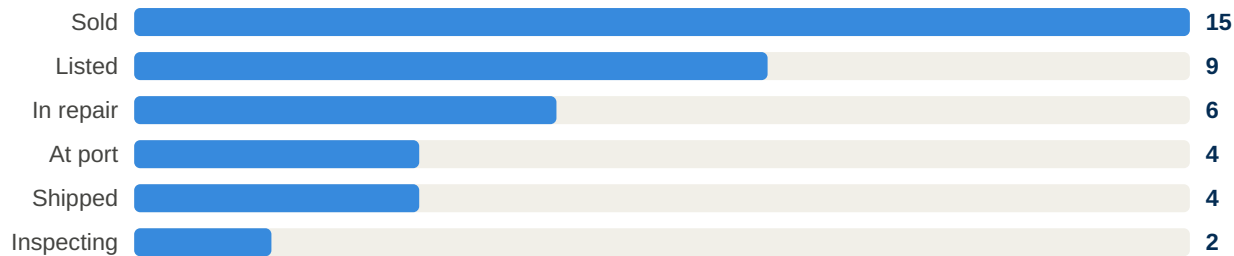
Data model

The Gold layer follows a standard star schema: **dim_vehicle** (tracked with slow-changing-dimension structure), **dim_customer**, **dim_staff**, **dim_branch**, **dim_auction_house**, **dim_vendor**, **dim_part**, **dim_date**, **dim_data_source**, and **dim_payment_method** surround **fact_purchase**, **fact_repair**, **fact_sale**, **fact_vehicle_lifecycle** (an accumulating snapshot tracking each vehicle end to end), **fact_profit**, and an append-only **fact_profit_history** for audit trail.

Business intelligence snapshot

The Gold layer already answers the questions a dealership owner or investor would ask first: how much inventory is in each stage, how profitable is it, and how fast does it move.

Inventory funnel (n = 40 vehicles)



Financial summary

Metric	Value
Total revenue (15 sold vehicles)	₦145,513,395
Total cost (purchase + repair)	₦110,913,153
Total profit	₦34,600,242
Average margin	22.91%
Average days, purchase to sale	81.1 days
Open data-quality review items	4 flagged

Cost currently covers purchase and repair only; shipping, customs, and cleaning overhead are not yet itemized (see roadmap, next page).

Data quality by design

Every automated extraction — from WhatsApp or receipts — carries a confidence score. Anything below threshold lands in a review queue instead of being silently accepted or dropped, so the dashboard's numbers stay trustworthy even when the inputs are messy. The build surfaced one real gap during testing: an ambiguous message ("that Toyota is looking good after the wash") slipped past the keyword classifier untagged — a concrete argument for why this review-queue pattern, not blind automation, belongs in production.

Engineering rigor

Six distinct issues were hit and resolved during the build, each documented with root cause. The most significant:

Cross-notebook table visibility (multi-day investigation)

Tables written in one notebook weren't visible from another. Two plausible-looking fixes (metastore path config, then a concurrency/locking theory) both failed. Root cause: no metastore was configured at all, so Spark's default in-memory catalog was scoped to a single kernel. Fixed by reading and writing Delta tables directly by file path — sidestepping the catalog entirely, since a Delta table's transaction log is self-describing.

Dependency resolution pulling a redundant 450MB download

Installing delta-spark via pip tried to fetch its own copy of PySpark, unaware the base Docker image already shipped one as a raw binary. Fixed with a targeted `--no-deps` install.

Schema inference failure on all-null columns

Spark can't infer a type with zero non-null samples. Fixed by supplying explicit schemas wherever nullable columns exist, applied proactively across the rest of the build.

Full chronology, including the two remaining issues (a random-seed reproducibility quirk, and a missing staff-role assignment), is in BUILD_LOG.md in the project repository.

Roadmap

Next step	Value
Migrate to Databricks (Unity Catalog)	Replaces the local path-based workaround with a real always-on catalog; multi-user access
Add expense tracking (shipping, customs, cleaning)	Completes true landed cost and margin accuracy
Replace rule-based agents with LLM extraction	Higher accuracy on ambiguous WhatsApp messages and receipt OCR
Build remaining five Power BI dashboards	Inventory, Workshop, Sales, Finance, and Driver/Salesman performance views
Connect live WhatsApp Business API and a receipt-capture app	Moves the platform from simulated to live intake

Why this is a credible foundation

The platform was built the way a production system would be — layered storage, explicit schemas, idempotent writes, and a documented incident trail rather than a black box — while staying small enough to build, break, and fix end to end in a portfolio timeline. That combination is the point: it shows both the ability to design a data platform and the discipline to debug one honestly when it doesn't work the first time.